
# National University of Computer and Emerging Sciences

Laboratory Manuals

for

# Computer Networks - Lab

(CL -3001)

**Department of Computer Science**

FAST-NU, Lahore, Pakistan

**Instructor:**

Muhammad Nouman Hanif

---

# Lab Manual 05

---

## Objective:

To understand and implement client-server communication using UDP Socket Programming.

## Introduction to Socket Programming

Socket programming is a way of connecting two nodes on a network to communicate. One socket (node) listens on a particular port at an IP address, while another socket reaches out to send a message. Sockets provide a form of Inter-Process Communication (IPC) and are the endpoints of a communications channel. The most common type of socket applications are client-server applications.

## UDP (User Datagram Protocol):

UDP uses a simple connectionless transmission model with a minimum protocol mechanism. With UDP, computer applications can send messages, referred to as datagrams, to other hosts on an Internet Protocol (IP) network without prior communications to set up special transmission channels or data paths. Following are its characteristics:

1. There is no guarantee of delivery, ordering, or duplicate protection.
2. Ideal for network applications where we can tolerate data loss but want to achieve low latency.

**Goal:** To learn how to build client/server applications that communicate using sockets.

**Socket:** A socket is a door between an application process and the end-to-end transport protocol (e.g., UDP or TCP). It serves as an endpoint for sending or receiving data across a computer network.

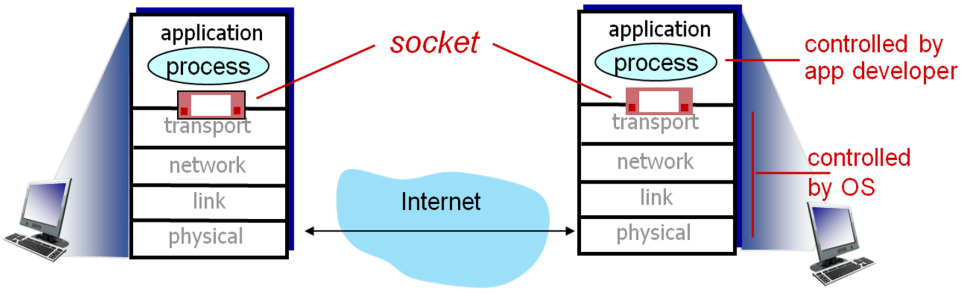


Figure 1: Socket programming model, illustrating the interface between the application and transport layers.

### Key Concepts: Connectionless vs. Connection-Oriented

- **TCP (Transmission Control Protocol):** This is a **connection-oriented** protocol. It establishes a reliable connection (like a phone call) before sending data, which guarantees delivery but adds overhead.
- **UDP (User Datagram Protocol):** This is a **connectionless** protocol, which is the focus of this lab. It's like sending a postcard: you send the data without establishing a prior

connection. It is fast and simple, but there is no guarantee of delivery or ordering.

## Client/Server UDP Interaction

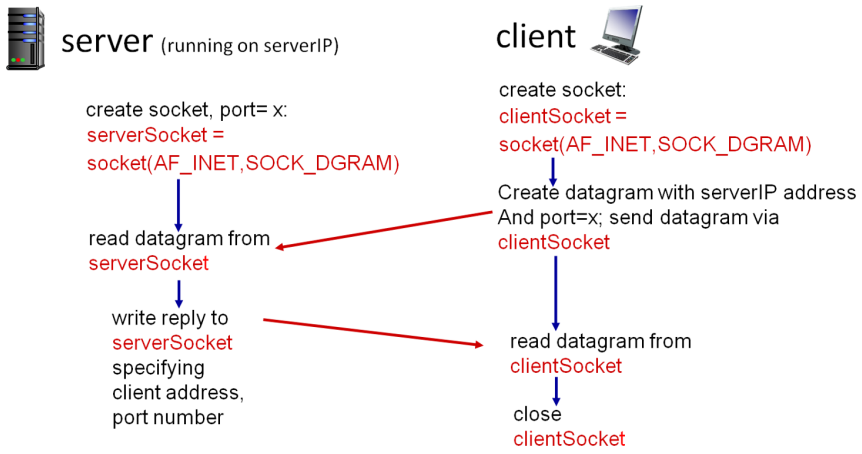


Figure 2: Client/Server socket interaction using UDP.

## Example: Python UDP Client and Server

*Python UDPClient*

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

include Python's socket library
create UDP socket
get user keyboard input
attach server name, port to message; send into socket
read reply data (bytes) from socket
print out received string and close socket

Figure 3: Example Python UDP Client.

*Python UDPServer*

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

create UDP socket
bind socket to local port number 12000
loop forever
Read from UDP socket into message, getting client's address (client IP and port)
send upper case string back to this client

Figure 4: Example Python UDP Server.

# Python Socket Programming Examples

## 1. Getting Your Local IP Address

```python
# Importing the socket module
import socket

# Getting the hostname
hostname = socket.gethostname()

# Getting the IP address
ip_address = socket.gethostbyname(hostname)

print(f"Hostname: {hostname}")
print(f"IP Address: {ip_address}")
```

## 2. Getting the IP Address of a Host

```python
import socket
host = "www.google.com"
ip_address = socket.gethostbyname(host)
print(f"Hostname: {host}")
print(f"IP Address: {ip_address}")
```

## 3. Getting the Hostname from an IP Address

```python
import socket
# Getting the hostname by using the IP address
hostname_info = socket.gethostbyaddr('8.8.8.8')
print(f"Details for IP 8.8.8.8: {hostname_info}")
```

## 4. Getting Service Name from Port and Protocol

```python
import socket
protocol = 'udp' # Changed to udp for relevance
ports = [53, 123, 161]
for port in ports:
    try:
        service_name = socket.getservbyport(port, protocol)
        print(f"Port {port} ({protocol}) => Service: {service_name}")
    except OSError:
        print(f"Port {port} ({protocol}) => Service not found")
```

## 5. Port Scanner (TCP)

*Note: This tool uses TCP sockets to check for open ports. It can take several minutes to run.*

```python
from socket import *
import time

startTime = time.time()

if __name__ == '__main__':
    target = input("Enter the hostname to be scanned: ")
    target_ip = gethostbyname(target)
    print("Starting scan on host: ", target_ip)
```

```
10
11      for i in range(50, 500):
12          s = socket(AF_INET, SOCK_STREAM)
13          conn = s.connect_ex((target_ip, i))
14          if conn == 0:
15              print("Port %d: OPEN" % (i,))
16          s.close()
17
18      print("Time Taken: ", time.time() - startTime, "seconds")
```

## Simple Client-Server Program (UDP Example)

This example demonstrates a basic UDP client and server that echo messages. The server listens for a message and sends the exact same message back to the client.

### Server Script (‘UDP_Server.py‘)

```
1  import socket
2
3  def main():
4      # Create a UDP socket
5      sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6
7      # Bind the socket to the server address and port
8      server_address = ('127.0.0.1', 2000)
9      sock.bind(server_address)
10
11     print("Socket created and bound")
12     print("Listening for messages...\n")
13
14     while True:
15         try:
16             # Receive the message from the client
17             client_message, client_address = sock.recvfrom(2000)
18             print(f"Received message from IP: {client_address[0]} and Port No: {
       client_address[1]}")
19             print(f"Client Message: {client_message.decode()}")
20
21             # Send the message back to the client
22             sock.sendto(client_message, client_address)
23
24         except Exception as e:
25             print(f"An error occurred: {e}")
26             break
27
28     # Closing the socket
29     sock.close()
30
31 if __name__ == "__main__":
32     main()
```

### Client Script (‘UDP_Client.py‘)

```python
import socket

def main():
    # Create a UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    server_address = ('127.0.0.1', 2000)

    try:
        # Get input from the user
        client_message = input("Enter Message: ")

        # Send the message to the server
        sock.sendto(client_message.encode(), server_address)

        # Receive the response from the server
        server_message, _ = sock.recvfrom(2000)
        print(f"Server Message: {server_message.decode()}")

    except Exception as e:
        print(f"An error occurred: {e}")

    finally:
        # Close the socket
        sock.close()

if __name__ == "__main__":
    main()
```

# Lab Exercises

## Task 1: Simple Chatbot using UDP Sockets

A university wants a simple chatbot server that answers basic student queries. Each query is a single transaction: a student sends a question and the server sends back an answer.

**Server Requirements:**

- Listen for incoming UDP packets.
- When a packet is received, check the message against a predefined set of questions.
- Respond to the client with the appropriate answer. If the question is not recognized, send back a default message like "Sorry, I don't understand that question."
- **Predefined Questions & Answers:**
  - "What is CGPA?" → "CGPA is the Cumulative Grade Point Average."
  - "What are the passing marks?" → "You need at least 50% to pass."
  - (Add 1-2 more of your own)

**Client Requirements:**

- Prompt the user to enter a question.
- Send the question as a UDP datagram to the server.
- Wait for the server's response and print it to the console.
- The client should terminate after receiving the answer. To ask another question, the client program must be run again.

## Task 2: GPA Calculator using UDP Sockets

A student can send their marks for a single subject to a server to calculate their GPA. The server processes the marks, assigns a grade and GPA, then returns the result to the student in a single UDP response.

**Server Requirements:**

- Listen for UDP packets containing a student's marks (a number from 0-100).
- Compute the grade and GPA based on the grading schema below.
- Send the result back to the student in a single message (e.g., "Grade: A, GPA: 4.00").
- If the input is not a valid number, send an error message.

**Client Requirements:**

- Prompt the user to enter their marks (out of 100).
- Send the marks to the server in a UDP datagram.

- Receive and display the grade and GPA from the server's response.

**Grading Schema:**

| BBA/BS | MS | Ph.D. | Equivalent % | Grade Points | Marks |
|---|---|---|---|---|---|
| A+ | A+ | A+ | 90 & Above | 4.00 | 90 & Above |
| A | A | A | 86 | 4.00 | 86 |
| A- | A- | A- | 82 | 3.67 | 82 |
| B+ | B+ | B+ | 78 | 3.33 | 78 |
| B | B | B | 74 | 3.00 | 74 |
| B- | B- | B- | 70 | 2.67* | 70 |
| C+ | C+ | F | 66 | 2.33* | 66 |
| C | C | | 62 | 2.00* | 62 |
| C- | F | | 58 | 1.67* | 58 |
| D+ | | | 54 | 1.33* | 54 |
| D | | | 50 | 1.00* | 50 |

*The grade point to be zero in case of grade "F" for MS/PhD student.

Figure 5: Grading Schema for Task 2

## Task 3: UDP-based Check-in/Check-out System

### Message Format

The client must send a message with the following format:

`YY-AAAA-CI` (For check-in, e.g., 20-4159-CI)
`YY-AAAA-CO` (For check-out, e.g., 20-4159-CO)

Where `YY-AAAA` is the student's roll number.

### Server Logic

The server must receive and process these packets. Your system must handle the following cases:

#### On "Check-in" ('-CI')

- If the student is not already checked in, the server marks their attendance in a student database (a Python list or dictionary is sufficient) and returns a welcome message: `"Welcome Student YY-AAAA"`.
- If the student is already checked in, it must send the message: `"You are already here."`.

#### On "Check-out" ('-CO')

- If the student is currently checked in, the server marks their out-attendance, removes them from the database, and sends the message: `"Goodbye Student YY-AAAA! Have a nice day."`.
- If a student sends a check-out packet without having checked in first, the server must return the message: `"You didn't check in today. Contact System Administrator."`.

**Important Notes:**

- The server must print the complete list of currently present students to its own console each time a client checks in or out.
- The server must not be restarted once started; it should handle multiple clients indefinitely.